

Amazon DynamoDB

Concepts, Data Model & Hands-on Lab based Learning

Course: DVS Technologies — AI Engineer with Cloud Foundations

Working Region for all labs: ap-south-1 (Asia Pacific — Mumbai)

Working dataset: batch-write.json — 10 retail purchase records

Parameter files: params/ — ready-made JSON arguments that work in any shell

Table used throughout: retail-purchases — partition key `customerid`, sort key `purchasedate`

How to use this guide

Modules 1–3 build the mental model; Module 4 is the hands-on lab you run yourself. Every example, product name, date, and query result in this guide was taken from the real batch-write.json file, so the outputs you see on your terminal will match the numbers printed here exactly. If they don't, something is wrong with your load step — go back to Lab 2.

Windows and macOS/Linux users

Every command in Module 4 is given in both forms: **macOS / Linux** first, then **Windows — Command Prompt**. Windows needs `^` instead of `\` to continue a line, and it has no single quotes, so JSON has to be double-quoted with escaped inner quotes. Read **Shell Compatibility** at the start of Module 4 before you type anything, and if you are in **PowerShell**, use the `params/` files described there.

Table of Contents

1. Module 1: Why NoSQL? The Mega Retail Platform Story
2. Module 2: What Is DynamoDB, and What Is an Item?
3. Module 3: Organising Data — Partition Keys, Sort Keys, Query vs. Scan
4. Module 4: Hands-On Lab — DynamoDB with the AWS CLI
5. Appendix A: The Dataset at a Glance
6. Appendix B: The Data File and a Complete Item
7. Appendix C: CLI Quick Reference

Module 1: Why NoSQL? The Mega Retail Platform Story

1.1 The scenario

Imagine you are managing the database for a massive online retail platform. Your store sells absolutely everything — from **Wireless Headphones** (Electronics) and **Cotton Casual Shirts** (Clothing) to **Data Engineering Handbooks** (Books) and **Garden Tool Sets** (Gardening).

Thousands of customers are buying completely different types of items every second. Every time a customer checks out, you need to save their order. At first glance this sounds like a standard database problem. But look closely at the *shape* of the data for just one order — `customer02` buying a book:

- It has ordinary fields: a product ID, a department, a price, a number of units.
- It has a **nested group** of product details: name, brand, subcategory.
- It has another nested group for the **delivery address**: city, state, postal code.
- It has a **list of tags**: `technology`, `programming`, `bestseller`.
- It has **two separate discounts applied at once**: `BOOK10` (10) and `MEMBER` (5).

That last point is the one that breaks a traditional design.

1.2 The SQL problem: a rigid spreadsheet

If we use a traditional SQL database such as **Amazon RDS**, the database behaves like a massive, rigid spreadsheet. It expects every order to have the exact same shape. That causes two headaches for our platform.

Headache 1 — wasted, meaningless columns. A *Cotton Casual Shirt* genuinely needs `size` and `colour`. A *Data Engineering Handbook* does not. In SQL you must create those columns anyway and fill them with blanks or N/A for millions of non-clothing orders.

Headache 2 — the JOIN problem. Look at `customer02`'s book order again: it used two discount codes. A relational row cannot hold a list inside a single cell. To save this one order you must write the order into an `Orders` table and then write **two more rows** into a separate `Discounts` table. When the customer opens their receipt, the database has to stitch those tables back together with a **JOIN**.

During a holiday sale, when thousands of people press *Buy* in the same second, your SQL database performs millions of these splits and stitches. Eventually it slows down, or it falls over.

1.3 The NoSQL answer: a pre-packaged box

This is where **NoSQL** — and specifically **Amazon DynamoDB** — changes the game. Instead of forcing your data into rigid rows spread across tables, DynamoDB stores each order as one flexible, self-contained document. It uses **Maps (M)** and **Lists (L)** to nest information *inside* the record.

The delivery address, the payment details, the tags, and both discounts all live inside the single order item. When `customer02` wants their receipt, DynamoDB does not consult separate `Address`, `Payment`, and `Discount` tables. It reaches in, grabs that one pre-packaged item, and returns it in milliseconds. **No JOIN required.**

Diagram 1.3 — The same order, stored two ways

SQL - one wide table, plus a second table for the discounts

productid	department	size	colour	price	
PROD-1003	Clothing	Large	Blue	55	<- columns used <- columns wasted
PROD-1002	Books	N/A	N/A	35	

productid	discount	amount	
PROD-1002	BOOK10	10	a JOIN is needed on every read
PROD-1002	MEMBER	5	

DYNAMODB - one self-contained item, one read, no JOIN

customerid	"customer02@example.com"	<- PARTITION KEY
purchasedate	"2026-01-07T11:30Z"	<- SORT KEY
productid	"PROD-1002"	
department	"Books"	
product	M { name, brand, subcategory }	
deliveryAddress	M { city, state, postalCode }	
payment	M { method, channel, reference }	
tags	SS [technology, programming, bestseller]	
discounts	L [{BOOK10, 10}, {MEMBER, 5}]	
ratings	NS [4, 5]	

1.4 SQL vs. NoSQL: the core differences

Feature	SQL (Relational, e.g. Amazon RDS)	NoSQL (DynamoDB)
---------	-----------------------------------	------------------

Feature	SQL (Relational, e.g. Amazon RDS)	NoSQL (DynamoDB)
Structure	Tables with fixed rows and columns	Flexible items; key-value and document
Analogy	A strict Excel spreadsheet	A filing cabinet of self-contained folders
Data shape	Flat and rigid. Nested data, such as two discounts, must be split into separate tables	Nested Maps , Lists , and Sets live inside one item
Empty fields	You maintain a <code>size</code> column even for Books and Televisions	You may store only the attributes that apply to that item
Reading data	Heavy JOINS stitch the record back together	Reads the whole pre-packaged item in one operation
Schema changes	Hard — you must <code>ALTER TABLE</code>	Easy — add attributes to new items on the fly
Best at	Complex queries, aggregations, strict relationships	Massive scale and predictable high-speed key lookups

1.5 The golden rule: RDS or DynamoDB?

Choose RDS (SQL) when:

- You need **complex analytics**. For example: *"total revenue for all Electronics, grouped by brand, where the customer used a WELCOME discount in January."* SQL is built for this. DynamoDB is genuinely bad at it.
- Your data is **highly connected** and strictly normalised, with many-to-many relationships.
- You need ad-hoc queries that you cannot predict in advance.

Choose DynamoDB (NoSQL) when:

- You need **extreme speed and scale** — a Black Friday sale where thousands of purchases per second must land without the database locking up.
- Your **access patterns are known and simple** — *"show me this customer's orders, newest first."*
- Your **data shape varies wildly** — you sell everything from 4K Smart Televisions to Ceramic Dinner Plate Sets, and each department carries different attributes.

The design rule that follows from this: in SQL you model the data first and write queries later. In DynamoDB you must know your **access patterns first**, because the keys you choose determine which questions are cheap to ask. Module 3 is where that becomes concrete.

Module 1 Key Takeaways

- SQL forces one rigid shape on every row; DynamoDB stores each item with its own shape.
- Nested data (two discounts on one order) needs extra tables and a **JOIN** in SQL; in DynamoDB it is a **List** inside the item.
- DynamoDB wins on scale and predictable lookups. RDS wins on analytics and relationships.
- **Know your access patterns before you design a DynamoDB table.**

Module 2: What Is DynamoDB, and What Is an Item?

2.1 What DynamoDB is

Amazon DynamoDB is AWS's fully managed **NoSQL** database. It is very fast, requires no servers or patching from you, and scales automatically whether you have 10 customers or 10 million. You interact with it purely through an API — from the console, the AWS CLI, or an SDK.

The vocabulary maps onto our story like this:

DynamoDB term	In our retail platform	Filing-cabinet analogy
Table	retail-purchases	The whole filing cabinet
Item	One purchase, e.g. PROD-1009 bought on 27 Jan	One sheet of paper
Attribute	department, price, tags	A field written on that sheet
Partition key	customerid	The label on a drawer
Sort key	purchasedate	The order of sheets inside that drawer

2.2 Anatomy of a real item

Below is one **complete, unmodified item** from `batch-write.json` — `customer01`'s denim jeans purchase. Read it slowly; every DynamoDB data type you need is visible in it.

```
{
  "customerid": { "S": "customer01@example.com" },
  "purchasedate": { "S": "2026-01-27T12:50Z" },
  "productid": { "S": "PROD-1009" },
  "department": { "S": "Clothing" },
  "size": { "S": "Medium" },
  "colour": { "S": "Black" },
  "units": { "N": "2" },
  "price": { "N": "75" },
  "weight": { "S": "Light" },
  "product": {
    "M": {
      "name": { "S": "Slim Fit Denim Jeans" },
      "brand": { "S": "DenimWorks" },
      "subcategory": { "S": "Mens Clothing" }
    }
  },
  "deliveryAddress": {
    "M": {
      "city": { "S": "Bengaluru" },
      "state": { "S": "Karnataka" },
      "postalCode": { "S": "560001" }
    }
  },
  "tags": { "SS": ["denim", "casual", "fashion"] },
  "payment": {
    "M": {
      "method": { "S": "UPI" },
      "channel": { "S": "MobileApp" },
      "reference": { "S": "TXN-90009" }
    }
  },
  "discounts": {
    "L": [
      { "M": { "type": { "S": "DENIM20" }, "amount": { "N": "20" } } }
    ]
  },
  "ratings": { "NS": ["3", "4", "5"] }
}
```

Decoding the magic:

- **Strings and numbers** are the ordinary building blocks: `{ "S": "Bengaluru" }, { "N": "75" }`. Note that **numbers are still written inside quotes** in the JSON wire format — DynamoDB converts them.
- **Maps (M)** — look at `product` and `deliveryAddress`. Rather than creating a whole `Addresses` table as SQL would demand, DynamoDB nests a mini-dictionary directly inside the order.
- **Sets (SS, NS)** — `tags` holds three unique strings, `ratings` holds unique numbers. In SQL, tracking multiple tags means another table and another join.
- **Lists (L)** — `discounts` is an ordered array of Maps, which is how one order carries two discount codes at once (see PROD-1002 and PROD-1006 in Appendix A).

Every order is fully self-contained.

2.3 The complete DynamoDB data type reference

Category	Type	JSON key	Description	Example
Scalar	String	"S"	Standard UTF-8 text. Cannot be empty in a key attribute.	{"S": "customer01@example.com"}
Scalar	Number	"N"	Integers or decimals, positive or negative. Always quoted in JSON.	{"N": "899"}
Scalar	Boolean	"BOOL"	true or false.	{"BOOL": true}
Scalar	Binary	"B"	Base64-encoded binary data.	{"B": "dGhpcyB0ZXh0..."}
Scalar	Null	"NULL"	An explicitly empty, missing, or unknown state.	{"NULL": true}
Document	Map	"M"	A nested dictionary (JSON object). May mix types and nest further.	{"M": {"city": {"S": "Bengaluru"}}
Document	List	"L"	An ordered array. May hold a mix of completely different types.	{"L": [{"S": "WELCOME"}, {"N": "50"}]}
Set	String Set	"SS"	Unordered array; every element must be a unique string.	{"SS": ["bestseller", "wireless"]}
Set	Number Set	"NS"	Unordered array of unique numbers.	{"NS": ["3", "4", "5"]}
Set	Binary Set	"BS"	Unordered array of unique binary values.	{"BS": ["U3Vubnk=", "UmFpbmk="]}

Two distinctions worth memorising. A **List** keeps its order and allows duplicates and mixed types; a **Set** has no order, allows no duplicates, and every element must share one type. And a **Map** is the nested structure you will reach for most often — it is what makes an item self-contained.

2.4 An honest word about N/A and truly absent attributes

DynamoDB's headline flexibility is that an item may simply **omit** an attribute that does not apply. It is worth noticing that our teaching dataset does **not** exercise that freedom: all 10 items carry all 15 attributes, and 7 of them store the literal string "N/A" in `size` (the book, the coffee maker, the television, and so on). One item, the book PROD-1002, stores "N/A" in `colour` too.

That is the very padding Module 1 criticised SQL for. It is fine for a uniform teaching file, but do not conclude that DynamoDB requires it. In a real design you would leave the attribute out entirely:

```
{
  "customerid": { "S": "customer02@example.com" },
  "purchasedate": { "S": "2026-01-07T11:30Z" },
  "productid": { "S": "PROD-1002" },
  "department": { "S": "Books" },
  "price": { "N": "35" }
}
```

No `size`. No `colour`. The item is smaller, cheaper to store, and honest about what it knows. The trade-off to remember: if an attribute may be absent, your application code must handle its absence, and a filter such as `size = :s` will silently skip items that lack the attribute altogether.

Module 2 Key Takeaways

- A **table** holds **items**; an item holds **attributes**; each attribute value is tagged with its type (S, N, M, L, SS, ...).
- **Maps** and **Lists** are what let one item replace three SQL tables.

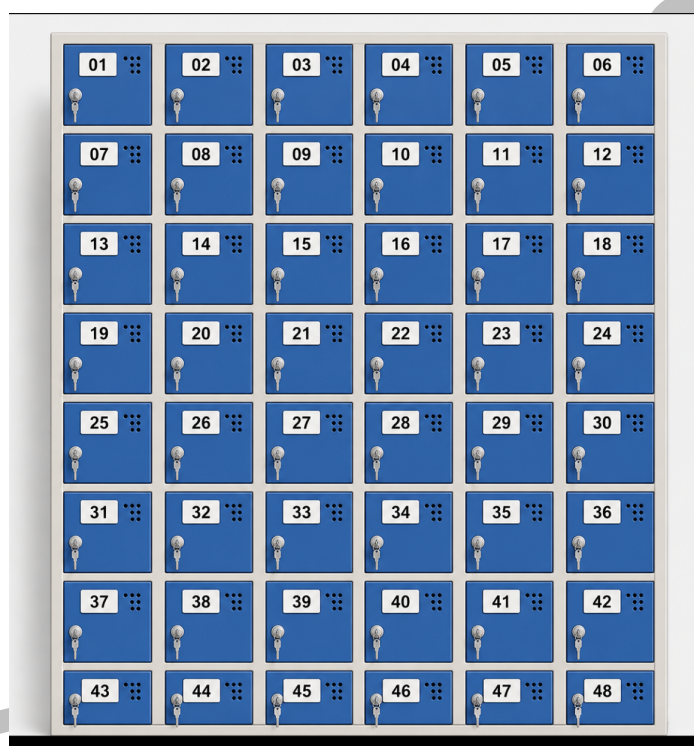
- **Sets** demand unique values of a single type; **Lists** are ordered and may mix types.
- Numbers travel as quoted strings in the JSON wire format.
- Omitting an attribute is allowed and often better than storing "N/A".

Module 3: Organising Data — Partition Keys, Sort Keys, Query vs. Scan

A **Scan** can find anything, but it opens every box in the database. With millions of purchases that is slow and expensive. DynamoDB therefore requires a **primary key** so it can organise data for fast retrieval. For our dataset the primary key has two parts: a **partition key** and a **sort key**.

3.1 The partition key: which drawer

Think of the table as a filing cabinet. The **partition key (PK)** is the label on the outside of a drawer; it groups related items together. DynamoDB hashes the key to decide which physical partition stores the item.



A bank of labelled lockers, each drawer holding the items for one key

In our dataset the natural partition key is **customerid**. Every purchase belonging to one customer lands in that customer's own drawer:

Drawer (customerid)	Items in that drawer
customer01@example.com	PROD-1001, PROD-1003, PROD-1009
customer02@example.com	PROD-1002, PROD-1006
customer03@example.com	PROD-1004, PROD-1008
customer04@example.com	PROD-1005
customer05@example.com	PROD-1007
customer06@example.com	PROD-1010

The benefit: when **customer01** opens their order history, DynamoDB does not read the whole company's sales. It walks straight to that one drawer and reads only those three items.

Design warning — hot partitions. A good partition key spreads traffic evenly. `customerid` does that well. A key such as `department` would not: with only six departments, *Electronics* would collect a disproportionate share of all reads and writes and become a bottleneck. Prefer keys with **high cardinality**.

3.2 The sort key: the order inside the drawer

Once `customer01`'s drawer is open, we find three purchases. The **sort key (SK)** fixes the order in which they sit, so a range of them can be read without inspecting the rest.

Our sort key is `purchasedate`, so DynamoDB keeps each customer's purchases in strict chronological order:

- `2026-01-05T09:15Z` — `PROD-1001`, Wireless Headphones
- `2026-01-10T14:20Z` — `PROD-1003`, Cotton Casual Shirt
- `2026-01-27T12:50Z` — `PROD-1009`, Slim Fit Denim Jeans

Because the timestamps are stored as **ISO-8601 strings**, alphabetical ordering and chronological ordering are the same thing — which is exactly why this format is used. `2026-01-05` sorts before `2026-01-10` as text just as it does in time.

Together, `customerid` + `purchasedate` form the **composite primary key**, and that pair must be unique for every item. In our file all 10 combinations are unique, so no item overwrites another.

Diagram 3.2 — The cabinet, drawn from the real data

```
TABLE retail-purchases
PARTITION KEY customerid -> which drawer
SORT KEY      purchasedate -> the order inside that drawer
```

customer01@example.com			
2026-01-05T09:15Z	PROD-1001	Wireless Headphones	
2026-01-10T14:20Z	PROD-1003	Cotton Casual Shirt	
2026-01-27T12:50Z	PROD-1009	Slim Fit Denim Jeans	

customer02@example.com			
2026-01-07T11:30Z	PROD-1002	Data Engineering Handbook	
2026-01-18T13:05Z	PROD-1006	Smart LED Monitor	

customer03@example.com			
2026-01-12T16:45Z	PROD-1004	Electric Coffee Maker	
2026-01-24T18:25Z	PROD-1008	4K Smart Television	

customer04@example.com			
2026-01-15T10:10Z	PROD-1005	Performance Running T-Shirt	

customer05@example.com			
2026-01-21T15:40Z	PROD-1007	Ceramic Dinner Plate Set	

customer06@example.com			
2026-01-30T17:15Z	PROD-1010	Garden Tool Set	

Diagram 3.3 — Why the bill differs

```
QUERY  customerid = customer01@example.com
       AND purchasedate > 2026-01-09T00:00Z

├─ goes straight to 1 drawer out of 6
├─ items are date-ordered, so it starts reading at 2026-01-10
├─ and never touches the 2026-01-05 headphones
└─ RETURNS  PROD-1003, PROD-1009          READS 2 items, BILLS 2 items

SCAN   department = "Home & Kitchen"

├─ opens all 6 drawers and reads all 10 items
├─ THEN applies the filter and throws most of them away
└─ RETURNS  PROD-1004, PROD-1007          READS 10 items, BILLS 10 items
```

The point students miss: `--filter-expression` runs **after** the read, not instead of it. A Scan that returns 2 items out of a 10-million-item table still reads — and charges for — 10 million items. Filters reduce what you *see*, never what you *pay*.

3.4 A gotcha: reserved words

`size` and `name` are on DynamoDB's **reserved word** list, and both appear in our data (`size` at the top level, `name` inside the product map). Using them directly in an expression fails:

macOS / Linux (bash or zsh):

```
# FAILS: Invalid FilterExpression: Attribute name is a reserved keyword; reserved keyword: size
aws dynamodb scan \
  --table-name retail-purchases \
  --filter-expression "size = :s" \
  --expression-attribute-values '{"s":{"S":"Medium"}}'
```

Windows — Command Prompt (cmd.exe):

```
REM FAILS: Invalid FilterExpression: Attribute name is a reserved keyword; reserved keyword: size
aws dynamodb scan ^
  --table-name retail-purchases ^
  --filter-expression "size = :s" ^
  --expression-attribute-values "{\"s\":{\"S\":\"Medium\"}}"
```

The fix is an **attribute-name placeholder**, which begins with `#`:

macOS / Linux (bash or zsh):

```
# WORKS
aws dynamodb scan \
  --table-name retail-purchases \
  --filter-expression "#s = :s" \
  --expression-attribute-names '{"#s":"size"}' \
  --expression-attribute-values '{"s":{"S":"Medium"}}'
```

Windows — Command Prompt (cmd.exe):

```
REM WORKS
aws dynamodb scan ^
  --table-name retail-purchases ^
  --filter-expression "#s = :s" ^
  --expression-attribute-names "{\"#s\":\"size\"}" ^
  --expression-attribute-values "{\"s\":{\"S\":\"Medium\"}}"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb scan --table-name retail-purchases --filter-expression "#s = :s" --expression-attribute-names
file://params/names-size.json --expression-attribute-values file://params/size-medium.json
```

Expected result: `PROD-1005` (Performance Running T-Shirt) and `PROD-1009` (Slim Fit Denim Jeans) — the only two items with `size = Medium`.

3.5 When the key is not enough: secondary indexes

Suppose someone asks: *"pull up the order for transaction `TXN-90008`."* With our current table that is **not** a Query, because TXN-90008 is neither the partition key nor the sort key — it is buried inside the payment map. Your only options are:

1. **A Scan with a filter** — correct, but it reads the whole table.
2. **A get-item, if you know the full key** — instant, but you must already know the customer and the timestamp.
3. **A Global Secondary Index (GSI)** — the proper answer for a recurring access pattern.

A GSI is a second view of the table with a different partition key, kept in sync automatically. One important limitation: **an index key must be a top-level scalar attribute**. You cannot index `payment.reference` while it is nested inside a map — you would first have to promote the transaction reference to a top-level attribute such as `txnref.productid`, by contrast, is already top-level and can be indexed directly:

macOS / Linux (bash or zsh):

```
aws dynamodb update-table \
  --table-name retail-purchases \
  --attribute-definitions AttributeName=productid,AttributeType=S \
  --global-secondary-index-updates '[{"Create":{"IndexName":"productid-
index","KeySchema":[{"AttributeName":"productid","KeyType":"HASH"}], "Projection":{"ProjectionType":"ALL"},"Pro
visionedThroughput":{"ReadCapacityUnits":5,"WriteCapacityUnits":5}}]'
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb update-table ^
  --table-name retail-purchases ^
  --attribute-definitions AttributeName=productid,AttributeType=S ^
  --global-secondary-index-updates "[{"Create\":{\"IndexName\":\"productid-
index\",\"KeySchema\":[{\"AttributeName\":\"productid\",\"KeyType\":\"HASH\"}],\"Projection\":{\"ProjectionTyp
e\":\"ALL\"},\"ProvisionedThroughput\":{\"ReadCapacityUnits\":5,\"WriteCapacityUnits\":5}}]"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb update-table --table-name retail-purchases --attribute-definitions
AttributeName=productid,AttributeType=S --global-secondary-index-updates file://params/gsi-productid.json
```

Creating a GSI is optional for this lab and consumes extra write capacity and storage. If you create one, remember it is deleted along with the table in the clean-up step.

Module 3 Key Takeaways

- **Partition key** = which drawer. Choose **high cardinality** (customerid, not department) to avoid hot partitions.
- **Sort key** = the order within the drawer. ISO-8601 timestamps sort chronologically as text, which makes date ranges cheap.
- `customerid + purchasedate` is the **composite primary key**; the pair must be unique.
- **Query** needs the partition key and reads only that partition. **Scan** reads everything, then filters.
- Filters reduce the rows you see, **not** the capacity you are billed for.
- Alias reserved words (size, name) with # placeholders.
- To query by a non-key attribute regularly, build a **GSI** — and its key must be top-level, not nested.

Module 4: Hands-On Lab — DynamoDB with the AWS CLI

Goal: create `retail-purchases`, load the 10 records, then retrieve them with Scan, Query, and GetItem.

Cost: effectively zero. DynamoDB's always-free allowance is **25 GB of storage plus 25 provisioned WCUs and 25 provisioned RCUs**. This lab uses 10 tiny items and 5 + 5 capacity units. Follow the clean-up step anyway.

Shell Compatibility — Read This First

Every command in this module is given twice: once for **macOS / Linux** (bash or zsh) and once for **Windows Command Prompt** (cmd.exe). The AWS CLI itself is identical on all platforms — what differs is how your **shell** handles line breaks and quotation marks. Three differences cause almost every "the command doesn't work on Windows" report.

	macOS / Linux (bash, zsh)	Windows Command Prompt (cmd.exe)	Windows PowerShell
Continue a command on the next line	<code>\</code> (backslash)	<code>^</code> (caret)	<code>` `</code> (backtick)
Wrap a JSON argument	<code>'single quotes'</code>	"double quotes", with every inner " written as <code>\</code>	see the note below
Comment a line	<code># comment</code>	<code>REM comment</code>	<code># comment</code>
Set an environment variable	<code>export AWS_PAGER=""</code>	<code>set AWS_PAGER=</code>	<code>\$env:AWS_PAGER=""</code>
Delete a file	<code>rm -f ~/.aws/credentials</code>	<code>del "%USERPROFILE%\aws\credentials"</code>	<code>Remove-Item \$HOME\aws\credentials</code>

The three rules for Command Prompt:

1. **Swap `\` for `^`** at the end of every continued line.
2. **cmd.exe has no single quotes.** Wrap JSON in double quotes and escape each inner quote: `'{"units":{"N":"3"}}'` becomes `"{\"units\":{\"N\":\"3\"}}"`.
3. **Never drop the quotes around a filter expression.** In cmd.exe, `>` means *redirect output to a file*. Typing `--filter-expression units > :units` unquoted will create a file called `:units` instead of filtering — and produce no error to tell you so. Keep `"units > :units"` quoted and it is treated as text.

⚠️ **PowerShell users, read this.** PowerShell's handling of quotes inside arguments to native programs **changed between Windows PowerShell 5.1 and PowerShell 7.3+**, so no single inline JSON form works on both. Check your version with `$PSVersionTable.PSVersion`, then pick one of these two reliable options:

- **Recommended — use a parameter file** (see the table below). It works unchanged in bash, cmd.exe, and every PowerShell version, because no quoting survives to be mangled.
- **Or use the stop-parsing token `--%`**, which tells PowerShell to pass everything after it through untouched, exactly like cmd.exe. Put it straight after `aws`, keep the command on one line, and use the Command Prompt escaping:

```

powershell
aws --% dynamodb scan --table-name retail-purchases --filter-expression "department = :dept" --expression-attribute-values "{\"dept\":{\"S\":\"Home & Kitchen\"}}"

```

The bulletproof option: parameter files

Any `--expression-attribute-values`, `--expression-attribute-names`, or `--key` argument can be read from a file with the `file://` prefix. This sidesteps shell quoting entirely and is the form to prefer in scripts on every operating system. The `params/` folder supplied with this course contains all of them, and **every lab command below is also given in this form**, labelled *Any OS — parameter file*:

```

aws dynamodb scan \
  --table-name retail-purchases \
  --filter-expression "department = :dept" \
  --expression-attribute-values file://params/dept-home-kitchen.json

```

That exact command works, character for character, in bash, Command Prompt, and PowerShell.

Parameter file	Contents	Used by
params/dept-home-kitchen.json	<code>{"dept":{"S":"Home & Kitchen"}}</code>	Lab 3a
params/units-3.json	<code>{"units":{"N":"3"}}</code>	Lab 3b
params/dept-electronics.json	<code>{"dept":{"S":"Electronics"}}</code>	Lab 3c
params/customer01.json	<code>{"customerid":{"S":"customer01@example.com"}}</code>	Lab 4a
params/customer01-jan.json	the :customerid, :start and :end values	Lab 4b
params/customer01-after.json	the :cid and :date values	Lab 4c
params/customer01-cid.json	<code>{"cid":{"S":"customer01@example.com"}}</code>	Lab 4d
params/channel-mobileapp.json	<code>{"channel":{"S":"MobileApp"}}</code>	Lab 5
params/brand-soundmax.json	<code>{"brand":{"S":"SoundMax"}}</code>	Lab 5
params/tag-bestseller.json	<code>{"tag":{"S":"bestseller"}}</code>	Lab 5
params/key-prod1008.json	the full primary key of PROD-1008	Lab 6
params/size-medium.json + params/names-size.json	the reserved-word example values and names	Module 3.4
params/gsi-productid.json	the productid-index definition	Module 3.5

File paths on Windows

file:// paths are resolved **relative to your current directory**, so cd into the folder holding batch-write.json before loading it:

```
cd C:\Users\%USERNAME%\Downloads\DynamoDB
aws dynamodb batch-write-item --request-items file://batch-write.json
```

An absolute Windows path also works, backslashes and drive letter included:

file://C:\Users\Student\Downloads\batch-write.json.

Turning off the output pager

Long JSON responses open in a pager on every platform. To disable it for one command, append `--no-cli-pager`. To disable it for the whole session:

Shell	Command
bash / zsh	<code>export AWS_PAGER=""</code>
Command Prompt	<code>set AWS_PAGER=</code>
PowerShell	<code>\$env:AWS_PAGER=""</code>

Lab 0 — Credentials and the AWS CLI

Step 1: Create an IAM user for CLI access (not the root user)

 **Do not create root user access keys.** The AWS console itself warns that *"root user access keys have long-term unlimited permissions that can't be restricted."* A leaked root key means the entire account — billing, IAM, every resource — with no way to limit the damage. Create an IAM user instead. It takes the same two minutes and the key can be scoped and revoked.

1. Sign in to the AWS console and open **IAM**.

2. Go to **Users** → **Create user**, name it `dynamodb-lab-user`, and click **Next**.
3. On **Set permissions**, choose **Attach policies directly** and select **AmazonDynamoDBFullAccess**.
 - For a stricter setup, attach a customer-managed policy limited to your one table ARN — this is the least-privilege habit from the IAM module.
4. Create the user, then open it and go to the **Security credentials** tab.
5. Under **Access keys**, choose **Create access key** → **Command Line Interface (CLI)**, acknowledge the recommendation, and click **Create access key**.
6. **Copy the Access key ID and Secret access key now**. The secret is shown exactly once. Store it in a password manager — never in a document, a screenshot, a chat message, or a Git repository.

Step 2: Install and configure the AWS CLI

Download the installer for your operating system from the AWS documentation:

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

Confirm the installation in a terminal or Command Prompt:

macOS / Linux (bash or zsh):

```
aws --version
# aws-cli/2.36.22 Python/3.14.6 Darwin/25.5.0 exe/arm64
```

Windows — Command Prompt (cmd.exe):

```
aws --version
REM aws-cli/2.36.22 Python/3.14.6 Windows/10 exe/AMD64
```

Now configure your credentials:

```
aws configure
```

Identical on Windows — nothing to change.

Answer the four prompts:

Prompt	Value to enter
AWS Access Key ID	the key ID you just copied
AWS Secret Access Key	the secret you just copied
Default region name	<code>ap-south-1</code>
Default output format	press Enter to accept <code>json</code>

Verify that the credentials work before going further:

```
aws sts get-caller-identity
```

Identical on Windows — nothing to change.

It should print your account ID and the ARN of `dynamodb-lab-user`. If it errors, your keys or region are wrong — fix that now rather than debugging it later.

Where do these credentials live? In `~/.aws/credentials` on macOS and Linux, or `%USERPROFILE%\aws\credentials` on Windows, in **plain text**. Treat that file as a password. On a shared or lab machine, delete it when you finish (see clean-up).

Lab 1 — Create the table

The table needs `customerid` as the partition key (HASH) and `purchasedate` as the sort key (RANGE). Note that you declare **only the key attributes** in `--attribute-definitions` — the other 13 attributes need no declaration at all, which is the schema flexibility from Module 1 in action.

macOS / Linux (bash or zsh):

```
aws dynamodb create-table \  
  --table-name retail-purchases \  
  --attribute-definitions \  
    AttributeName=customerid,AttributeType=S \  
    AttributeName=purchasedate,AttributeType=S \  
  --key-schema \  
    AttributeName=customerid,KeyType=HASH \  
    AttributeName=purchasedate,KeyType=RANGE \  
  --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5 \  
  --region ap-south-1
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb create-table ^  
  --table-name retail-purchases ^  
  --attribute-definitions ^  
    AttributeName=customerid,AttributeType=S ^  
    AttributeName=purchasedate,AttributeType=S ^  
  --key-schema ^  
    AttributeName=customerid,KeyType=HASH ^  
    AttributeName=purchasedate,KeyType=RANGE ^  
  --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5 ^  
  --region ap-south-1
```

Wait until the table is ready, then confirm its status:

macOS / Linux (bash or zsh):

```
aws dynamodb wait table-exists --table-name retail-purchases  
  
aws dynamodb describe-table \  
  --table-name retail-purchases \  
  --query 'Table.[TableName,TableStatus,KeySchema]'
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb wait table-exists --table-name retail-purchases  
  
aws dynamodb describe-table ^  
  --table-name retail-purchases ^  
  --query "Table.[TableName,TableStatus,KeySchema]"
```

Why PROVISIONED with 5 + 5? The always-free allowance covers 25 provisioned RCUs and WCUs, so 5 + 5 is comfortably inside it. **On-demand** (`--billing-mode PAY_PER_REQUEST`) is simpler and excellent in production, but it is billed per request rather than covered by the always-free capacity allowance. For a class exercise, provisioned is the cheaper habit.

Lab 2 — Load the 10 retail records

Place `batch-write.json` in your current directory and run:

```
aws dynamodb batch-write-item --request-items file://batch-write.json
```

Identical on Windows — nothing to change.

Check the response. A successful load returns an empty `UnprocessedItems` map:

```
{
  "UnprocessedItems": {}
}
```

batch-write-item fails quietly. If DynamoDB throttles some writes, the call still succeeds overall and lists the rejected items under `UnprocessedItems`. It is your job to inspect that field and resubmit. Also note the hard limit of **25 items per request** — our file has 10, so one call is enough.

Confirm all 10 items arrived:

macOS / Linux (bash or zsh):

```
aws dynamodb scan --table-name retail-purchases --select COUNT
# expect: "Count": 10, "ScannedCount": 10
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb scan --table-name retail-purchases --select COUNT
REM expect: "Count": 10, "ScannedCount": 10
```

You can also browse them in the console under **DynamoDB → Tables → retail-purchases → Explore table items**.

Navigating CLI output

Long JSON responses open in a pager. The keys are:

Key	Action
↓ / ↑	Scroll line by line
Space	Page down
b	Page back up
g	Jump to the beginning
G	Jump to the end
q	Quit the pager

To switch the pager off for a single command, append `--no-cli-pager`. To disable it for the whole session, use `export AWS_PAGER=""` on macOS or Linux, set `AWS_PAGER=` in Command Prompt, or `$env:AWS_PAGER=""` in PowerShell.

Lab 3 — Scan: search the whole table

A **Scan** reads every item and then applies your filter. Use it when you are searching on something that is not a key.

3a. Find purchases in one department

macOS / Linux (bash or zsh):

```
aws dynamodb scan \
  --table-name retail-purchases \
  --filter-expression "department = :dept" \
  --expression-attribute-values '{":dept":{"S":"Home & Kitchen"}}'
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb scan ^
--table-name retail-purchases ^
--filter-expression "department = :dept" ^
--expression-attribute-values "{\":dept\":{\":S\":\":Home & Kitchen\"}}"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb scan --table-name retail-purchases --filter-expression "department = :dept" --expression-attribute-values file://params/dept-home-kitchen.json
```

- `--filter-expression "department = :dept"` keeps only items in the chosen department.
- `--expression-attribute-values` supplies the real value for the `:dept` placeholder.

Expected result — 2 items: PROD-1004 (Electric Coffee Maker) and PROD-1007 (Ceramic Dinner Plate Set).

The department string must match the data exactly. The six valid values are **Books, Clothing, Electronics, Gardening, Home & Kitchen, Sports**. A near-miss such as "Home Appliances" is not an error — it simply returns `"Count": 0`, which is a confusing way to learn about typos.

3b. Find bulk purchases

macOS / Linux (bash or zsh):

```
aws dynamodb scan \
--table-name retail-purchases \
--filter-expression "units > :units" \
--expression-attribute-values '{":units":{\":N\":":3\"}}'
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb scan ^
--table-name retail-purchases ^
--filter-expression "units > :units" ^
--expression-attribute-values "{\":units\":{\":N\":\":3\"}}"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb scan --table-name retail-purchases --filter-expression "units > :units" --expression-attribute-values file://params/units-3.json
```

Expected result — 2 items: PROD-1005 (4 units) and PROD-1007 (5 units).

3c. Return only the fields you care about

macOS / Linux (bash or zsh):

```
aws dynamodb scan \
--table-name retail-purchases \
--filter-expression "department = :dept" \
--expression-attribute-values '{":dept":{\":S\":":Electronics\"}}' \
--projection-expression "productid, price, product.brand"
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb scan ^
--table-name retail-purchases ^
--filter-expression "department = :dept" ^
--expression-attribute-values '{"":dept\":{"S\":"Electronics\":"}}' ^
--projection-expression "productid, price, product.brand"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb scan --table-name retail-purchases --filter-expression "department = :dept" --expression-attribute-values file://params/dept-electronics.json --projection-expression "productid, price, product.brand"
```

Expected result — 3 items: PROD-1001, PROD-1006, PROD-1008. --projection-expression trims the response payload; it does **not** reduce the capacity consumed, because the items were already read.

Lab 4 — Query: use the keys

A **Query** must supply the partition key, which is why it is fast.

4a. All purchases by one customer

macOS / Linux (bash or zsh):

```
aws dynamodb query \
--table-name retail-purchases \
--key-condition-expression "customerid = :customerid" \
--expression-attribute-values '{"":customerid\":{"S\":"customer01@example.com\":"}}'
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb query ^
--table-name retail-purchases ^
--key-condition-expression "customerid = :customerid" ^
--expression-attribute-values '{"":customerid\":{"S\":"customer01@example.com\":"}}'
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb query --table-name retail-purchases --key-condition-expression "customerid = :customerid" --expression-attribute-values file://params/customer01.json
```

Expected result — 3 items in date order: PROD-1001 (5 Jan), PROD-1003 (10 Jan), PROD-1009 (27 Jan).

4b. Purchases within a date window

Because purchasedate is the sort key, it can be combined with the partition key to read a contiguous slice:

macOS / Linux (bash or zsh):

```
aws dynamodb query \
--table-name retail-purchases \
--key-condition-expression "customerid = :customerid AND purchasedate BETWEEN :start AND :end" \
--expression-attribute-values '{"":customerid\":{"S\":"customer01@example.com"}, ":start\":{"S\":"2026-01-01T00:00Z"}, ":end\":{"S\":"2026-01-31T23:59Z\":"}}'
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb query ^
--table-name retail-purchases ^
--key-condition-expression "customerid = :customerid AND purchasedate BETWEEN :start AND :end" ^
```

```
--expression-attribute-values "{\"cid\":\"customer01@example.com\"},
\":start\":{\"S\":\"2026-01-01T00:00Z\"}, \":end\":{\"S\":\"2026-01-31T23:59Z\"}}"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb query --table-name retail-purchases --key-condition-expression "customerid = :customerid AND
purchasedate BETWEEN :start AND :end" --expression-attribute-values file://params/customer01-jan.json
```

Expected result — all 3 of customer01's January purchases.

4c. Everything after a given date

macOS / Linux (bash or zsh):

```
aws dynamodb query \
--table-name retail-purchases \
--key-condition-expression "customerid = :cid AND purchasedate > :date" \
--expression-attribute-values '{"cid":{"S":"customer01@example.com"}, ":date":{"S":"2026-01-09T00:00Z"}}'
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb query ^
--table-name retail-purchases ^
--key-condition-expression "customerid = :cid AND purchasedate > :date" ^
--expression-attribute-values "{\"cid\":{\"S\":\"customer01@example.com\"}, \":date\":{\"S\":\"2026-01-09T00:00Z\"}}"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb query --table-name retail-purchases --key-condition-expression "customerid = :cid AND
purchasedate > :date" --expression-attribute-values file://params/customer01-after.json
```

Expected result — 2 items: PROD-1003 (Cotton Casual Shirt) and PROD-1009 (Slim Fit Denim Jeans). DynamoDB locates customer01's partition, and because the items are already sorted by date it begins reading at 10 January and never touches the 5 January headphones.

4d. Newest first, and just the latest order

macOS / Linux (bash or zsh):

```
aws dynamodb query \
--table-name retail-purchases \
--key-condition-expression "customerid = :cid" \
--expression-attribute-values '{"cid":{"S":"customer01@example.com"}}' \
--no-scan-index-forward \
--max-items 1
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb query ^
--table-name retail-purchases ^
--key-condition-expression "customerid = :cid" ^
--expression-attribute-values "{\"cid\":{\"S\":\"customer01@example.com\"}}" ^
--no-scan-index-forward ^
--max-items 1
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb query --table-name retail-purchases --key-condition-expression "customerid = :cid" --expression-attribute-values file://params/customer01-cid.json --no-scan-index-forward --max-items 1
```

`--no-scan-index-forward` reverses the sort-key order, so the newest purchase comes first: `PROD-1009`, bought 27 January. This is the standard "most recent order" pattern.

Lab 5 — Querying nested attributes

Maps are addressed with dot notation. Both examples below are **Scans**, because `payment.channel` and `product.brand` are not key attributes.

macOS / Linux (bash or zsh):

```
# Purchases made through the mobile app
aws dynamodb scan \
  --table-name retail-purchases \
  --filter-expression "payment.channel = :channel" \
  --expression-attribute-values '{"channel":{"S":"MobileApp"}}'
```

Windows — Command Prompt (cmd.exe):

```
REM Purchases made through the mobile app
aws dynamodb scan ^
  --table-name retail-purchases ^
  --filter-expression "payment.channel = :channel" ^
  --expression-attribute-values "{\"channel\":{\"S\":\"MobileApp\"}}"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb scan --table-name retail-purchases --filter-expression "payment.channel = :channel" --expression-attribute-values file://params/channel-mobileapp.json
```

Expected result — 5 items: `PROD-1001`, `PROD-1003`, `PROD-1005`, `PROD-1007`, `PROD-1009`. The other five were bought through Web.

macOS / Linux (bash or zsh):

```
# Purchases of a particular brand
aws dynamodb scan \
  --table-name retail-purchases \
  --filter-expression "product.brand = :brand" \
  --expression-attribute-values '{"brand":{"S":"SoundMax"}}'
```

Windows — Command Prompt (cmd.exe):

```
REM Purchases of a particular brand
aws dynamodb scan ^
  --table-name retail-purchases ^
  --filter-expression "product.brand = :brand" ^
  --expression-attribute-values "{\"brand\":{\"S\":\"SoundMax\"}}"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb scan --table-name retail-purchases --filter-expression "product.brand = :brand" --expression-attribute-values file://params/brand-soundmax.json
```

Expected result — 1 item: `PROD-1001`, the Wireless Headphones.

macOS / Linux (bash or zsh):

```
# Items carrying a particular tag inside a String Set
aws dynamodb scan \
  --table-name retail-purchases \
  --filter-expression "contains(tags, :tag)" \
  --expression-attribute-values '{"tag":{"S":"bestseller"}}'
```

Windows — Command Prompt (cmd.exe):

```
REM Items carrying a particular tag inside a String Set
aws dynamodb scan ^
  --table-name retail-purchases ^
  --filter-expression "contains(tags, :tag)" ^
  --expression-attribute-values "{\"tag\":{\"S\":\"bestseller\"}}"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb scan --table-name retail-purchases --filter-expression "contains(tags, :tag)" --expression-attribute-values file://params/tag-bestseller.json
```

Expected result — 2 items: PROD-1001 and PROD-1002. contains() is how you search inside a Set or a List.

Remember Module 3.4: filtering on product.name would fail, because name is a reserved word. You would need --expression-attribute-names '{"#n":"name"}' and the expression product.#n = :v.

Lab 6 — GetItem: the fastest read of all

If you know the complete primary key, skip Query and Scan entirely. **GetItem** fetches exactly one item.

macOS / Linux (bash or zsh):

```
aws dynamodb get-item \
  --table-name retail-purchases \
  --key '{"customerid":{"S":"customer03@example.com"}, "purchasedate":{"S":"2026-01-24T18:25Z"}}'
```

Windows — Command Prompt (cmd.exe):

```
aws dynamodb get-item ^
  --table-name retail-purchases ^
  --key "{\"customerid\":{\"S\":\"customer03@example.com\"}, \"purchasedate\":{\"S\":\"2026-01-24T18:25Z\"}}"
```

Any OS — parameter file (one line, no continuation character):

```
aws dynamodb get-item --table-name retail-purchases --key file://params/key-prod1008.json
```

Expected result — 1 item: PROD-1008, the 4K Smart Television, transaction TXN-90008. This is the cheapest and fastest operation DynamoDB offers, and it is why the primary key design in Module 3 matters so much.

Lab Clean-Up Steps — DO NOT SKIP

The 10 items in this lab cost effectively nothing, but leaving tables behind is a bad habit, and provisioned capacity is billed per hour once your account exceeds the free allowance across all tables.

Step 1 — Delete the table. This removes the items and any secondary indexes with it:

```
aws dynamodb delete-table --table-name retail-purchases
aws dynamodb wait table-not-exists --table-name retail-purchases
```

Identical on Windows — nothing to change.

Step 2 — Confirm nothing is left in this Region:

```
aws dynamodb list-tables --region ap-south-1
```

Identical on Windows — nothing to change.

Step 3 — Deactivate or delete the access key you created in Lab 0: **IAM** → **Users** → **dynamodb-lab-user** → **Security credentials** → **Access keys** → **Actions** → **Deactivate**, then **Delete**. An unused long-term key is a standing risk with no benefit.

Step 4 — On a shared or classroom machine, remove your local credentials:

macOS / Linux (bash or zsh):

```
rm -f ~/.aws/credentials
```

Windows — Command Prompt (cmd.exe):

```
del "%USERPROFILE%\aws\credentials"
```

Step 5 — Check the bill at **Billing and Cost Management** → **Free tier** the next day. Usage takes up to 24 hours to appear.

Module 4 Key Takeaways

- Create the table with only the **key attributes** declared; everything else is free-form.
- **batch-write-item** handles 25 items per call and reports rejects in **UnprocessedItems** — always check it.
- **Scan** for non-key searches, **Query** for key-based ones, **GetItem** when you know the whole key.
- **--projection-expression** shrinks the response, not the bill.
- **--no-scan-index-forward** gives you "newest first"; **contains()** searches inside Sets and Lists.
- Delete the table **and** the access key when you finish.

Appendix A: The Dataset at a Glance

All 10 items in `batch-write.json`, ordered by product ID. Use this as the answer key for every lab.

Product ID	Customer	Purchase date (SK)	Department	Product	Brand	Units	Price	Payment	Channel	Ref
PROD-1001	customer01	2026-01-05T09:15Z	Electronics	Wireless Headphones	SoundMax	1	899	UPI	MobileApp	TXN-90001
PROD-1002	customer02	2026-01-07T11:30Z	Books	Data Engineering Handbook	TechPress	2	35	CreditCard	Web	TXN-90002
PROD-1003	customer01	2026-01-10T14:20Z	Clothing	Cotton Casual Shirt	UrbanWear	3	55	UPI	MobileApp	TXN-90003
PROD-1004	customer03	2026-01-12T16:45Z	Home &	Electric Coffee	BrewPro	1	129	DebitCard	Web	TXN-90004

Product ID	Customer	Purchase date (SK)	Department	Product	Brand	Units	Price	Payment	Channel	Ref
			Kitchen	Maker						
PROD-1005	customer04	2026-01-15T10:10Z	Sports	Performance Running T-Shirt	RunFit	4	40	UPI	MobileApp	TXN-90005
PROD-1006	customer02	2026-01-18T13:05Z	Electronics	Smart LED Monitor	ViewMax	2	649	CreditCard	Web	TXN-90006
PROD-1007	customer05	2026-01-21T15:40Z	Home & Kitchen	Ceramic Dinner Plate Set	HomeCraft	5	25	UPI	MobileApp	TXN-90007
PROD-1008	customer03	2026-01-24T18:25Z	Electronics	4K Smart Television	VisionPlus	1	1199	CreditCard	Web	TXN-90008
PROD-1009	customer01	2026-01-27T12:50Z	Clothing	Slim Fit Denim Jeans	DenimWorks	2	75	UPI	MobileApp	TXN-90009
PROD-1010	customer06	2026-01-30T17:15Z	Gardening	Garden Tool Set	GreenGrow	3	32	DebitCard	Web	TXN-90010

Customer email addresses are shown short in the table above; the stored values are the full customerNN@example.com strings.

Product ID	size	colour	weight	City	tags (SS)	ratings (NS)	discounts (L)
PROD-1001	N/A	Black	Medium	Bengaluru	bestseller, wireless, audio	4, 5	WELCOME (50)
PROD-1002	N/A	N/A	Light	Mysuru	technology, programming, bestseller	4, 5	BOOK10 (10) + MEMBER (5)
PROD-1003	Large	Blue	Light	Bengaluru	cotton, casual, summer	3, 4, 5	FASHION15 (15)
PROD-1004	N/A	Silver	Medium	Chennai	coffee, kitchen, appliance	4, 5	KITCHEN20 (20)
PROD-1005	Medium	Green	Medium	Hyderabad	running, sportswear, fitness	4, 5	SPORTS10 (10)
PROD-1006	N/A	White	Heavy	Pune	monitor, office, work-from-home	3, 4, 5	ELECTRONICS25 (25) + MEMBER (15)
PROD-1007	N/A	Red	Light	Mumbai	dining, ceramic, home	4, 5	HOME15 (15)
PROD-1008	N/A	Grey	Heavy	Delhi	4K, smart-tv, entertainment	4, 5	TV50 (50)
PROD-1009	Medium	Black	Light	Bengaluru	denim, casual, fashion	3, 4, 5	DENIM20 (20)
PROD-1010	N/A	Green	Medium	Coimbatore	gardening, tools, outdoor	4, 5	GARDEN10 (10)

Useful counts for checking your query results

Question	Answer
Items in total	10
Distinct customers (partition keys)	6
Departments	6 — Electronics (3), Clothing (2), Home & Kitchen (2), Books (1), Sports (1), Gardening (1)
Payment channels	MobileApp (5), Web (5)
Payment methods	UPI (5), CreditCard (3), DebitCard (2)

Question	Answer
Items with two discounts	2 — PROD-1002, PROD-1006
Items with units > 3	2 — PROD-1005, PROD-1007
Items with size = "N/A"	7

Appendix B: The Data File and a Complete Item

File structure

batch-write.json is a **BatchWriteItem** request document, not a plain list of records. The top-level key is the **table name**, and each entry is wrapped in a **PutRequest**:

```
{
  "retail-purchases": [
    { "PutRequest": { "Item": { "...first item..." } } },
    { "PutRequest": { "Item": { "...second item..." } } }
  ]
}
```

Two consequences follow from that shape:

- **The table name is baked into the file.** If you create a table with a different name, the load will fail — the name in the file must match.
- **The file is capped at 25 items** per call, because that is the BatchWriteItem limit. Ours has 10.

A note on the file name. Earlier versions of these notes referenced file://retail-data.json. The file supplied with this course is named **batch-write.json**, so use that name — or rename the file to match your command. The path is resolved relative to your current working directory, so run the command from the folder containing the file.

One complete item, verbatim

This is the first record in the file, exactly as stored — **customer01**'s headphone purchase:

```
{
  "PutRequest": {
    "Item": {
      "customerid": { "S": "customer01@example.com" },
      "purchasedate": { "S": "2026-01-05T09:15Z" },
      "productid": { "S": "PROD-1001" },
      "department": { "S": "Electronics" },
      "size": { "S": "N/A" },
      "colour": { "S": "Black" },
      "units": { "N": "1" },
      "price": { "N": "899" },
      "weight": { "S": "Medium" },
      "product": {
        "M": {
          "name": { "S": "Wireless Headphones" },
          "brand": { "S": "SoundMax" },
          "subcategory": { "S": "Audio" }
        }
      }
    },
    "deliveryAddress": {
      "M": {
        "city": { "S": "Bengaluru" },
        "state": { "S": "Karnataka" },
        "postalCode": { "S": "560001" }
      }
    }
  }
}
```

```

    }
  },
  "tags": { "SS": ["bestseller", "wireless", "audio"] },
  "payment": {
    "M": {
      "method": { "S": "UPI" },
      "channel": { "S": "MobileApp" },
      "reference": { "S": "TXN-90001" }
    }
  },
  "discounts": {
    "L": [
      { "M": { "type": { "S": "WELCOME" }, "amount": { "N": "50" } } }
    ]
  },
  "ratings": { "NS": ["4", "5"] }
}
}
}

```

Appendix C: CLI Quick Reference

Every command assumes `--region ap-south-1` from your `aws` configure defaults. Commands marked **same** are byte-identical on macOS, Linux, and Windows because they contain no JSON and no line continuations.

Task	Command	Windows
Check the CLI version	<code>aws --version</code>	same
Configure credentials	<code>aws configure</code>	same
Confirm who you are	<code>aws sts get-caller-identity</code>	same
List tables	<code>aws dynamodb list-tables</code>	same
Wait until ready	<code>aws dynamodb wait table-exists --table-name retail-purchases</code>	same
Bulk load	<code>aws dynamodb batch-write-item --request-items file://batch-write.json</code>	same
Count items	<code>aws dynamodb scan --table-name retail-purchases --select COUNT</code>	same
Delete the table	<code>aws dynamodb delete-table --table-name retail-purchases</code>	same
Create the table	see Lab 1	swap <code>\</code> for <code>^</code>
Describe the table	see Lab 1	swap <code>\</code> for <code>^</code> , and <code>'...'</code> for <code>"..."</code>
Read one item by key	see Lab 6	escape the JSON, or use <code>file://params/key-prod1008.json</code>
Query one partition	see Lab 4	escape the JSON, or use <code>file://params/customer01.json</code>
Scan with a filter	see Lab 3	escape the JSON, or use <code>file://params/dept-home-kitchen.json</code>
Add one item	<code>aws dynamodb put-item --table-name retail-purchases -item file://item.json</code>	same, when the item is in a file
Delete one item	<code>aws dynamodb delete-item --table-name retail-purchases --key file://params/key-prod1008.json</code>	same, when the key is in a file

Shell syntax at a glance

	macOS / Linux	Command Prompt	PowerShell
Line continuation	<code>\</code>	<code>^</code>	<code>`</code> <code>''</code>
JSON quoting	<code>'{...}'</code>	<code>"{\"...\"}"</code>	use <code>file://</code> or <code>--%</code>
Comment	<code>#</code>	<code>REM</code>	<code>#</code>
Disable pager	<code>export AWS_PAGER=""</code>	<code>set AWS_PAGER=</code>	<code>\$env:AWS_PAGER=""</code>
Universal alternative	<code>file://params/<name>.json</code> works everywhere, in every shell		

Expression flag cheat sheet

Flag	Purpose	Placeholder
<code>--key-condition-expression</code>	Query only. Must include the partition key	<code>:value</code>
<code>--filter-expression</code>	Applied after the read, in Query or Scan	<code>:value</code>
<code>--expression-attribute-values</code>	Supplies the actual values	defines <code>:value</code>
<code>--expression-attribute-names</code>	Aliases reserved words such as <code>size</code> , <code>name</code>	defines <code>#name</code>
<code>--projection-expression</code>	Chooses which attributes come back	—

A final word

DynamoDB rewards you for deciding, up front, how your data will be read. Get the partition key and sort key right and your queries stay fast and cheap forever. Get them wrong and you will spend the rest of the project scanning. Practise on this 10-item table until Query, Scan, and GetItem feel obvious — the pattern is identical at ten items and at ten billion.

Happy building!